

Session 6: Loops (For, While, Do-While) Topics: Difference between entry-controlled and exit-controlled loops. Use cases (e.g., menu-driven programs). Example: Countdown timer using loops.

- 1) Write a program that prints the numbers from 10 down to 1 using a for loop, similar to a countdown timer.**

```
#include<stdio.h>
```

```
int main(){
```

```
    int i;
```

```
    printf("countdown timer:\n");
```

```
    for(i=10; i>=1; i--){
```

```
        printf("%d\n",i);
```

```
    }
```

```
    return 0;
```

```
}
```

Output -

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
@abhipatil1137 →/workspaces/6-jun-26/practical (master) $ ./a.out
```

```
10
9
8
7
6
5
4
3
2
1
```

- 2) Create a menu-driven console app that lets the user: 1) View your favorite 3 IPL teams, 2) Add a new team, 3) Exit. Use a while loop to keep showing the menu until the user chooses Exit.

Hint: Use input() (or Scanner in Java) to get the user's choice each time.

```
#include<stdio.h>
```

```
int main(){
```

```
    char teams[4][30]={
        "chanai super kings",
        "mumbai indians",
        "punjab kings"
    };
```

```
    int choice;
    int added=0;
```

```
    while (1)
    {
        printf("\n===== IPL new menu =====\n");
        printf("1. view the favorite ipl teams\n");
        printf("2. new ipl team added\n");
        printf("3. exit\n");
```

```
printf("enter the choice: ");
scanf("%d",&choice);

switch (choice)
{
case 1:
    printf("\nFevorite ipl teams");
    printf("1. %s\n",teams[0]);
    printf("2. %s\n",teams[1]);
    printf("3. %s\n",teams[2]);

    if(added)
    {
        printf("4. %s\n",teams[3]);
    }

    break;
case 2:
    if(!added){
        printf("enter the new ipl team\n");
        scanf("%s",teams[3]);
        added=1;
        printf("team added succsefull\n");
    } else{
        printf("only one team can be added\n");
    }

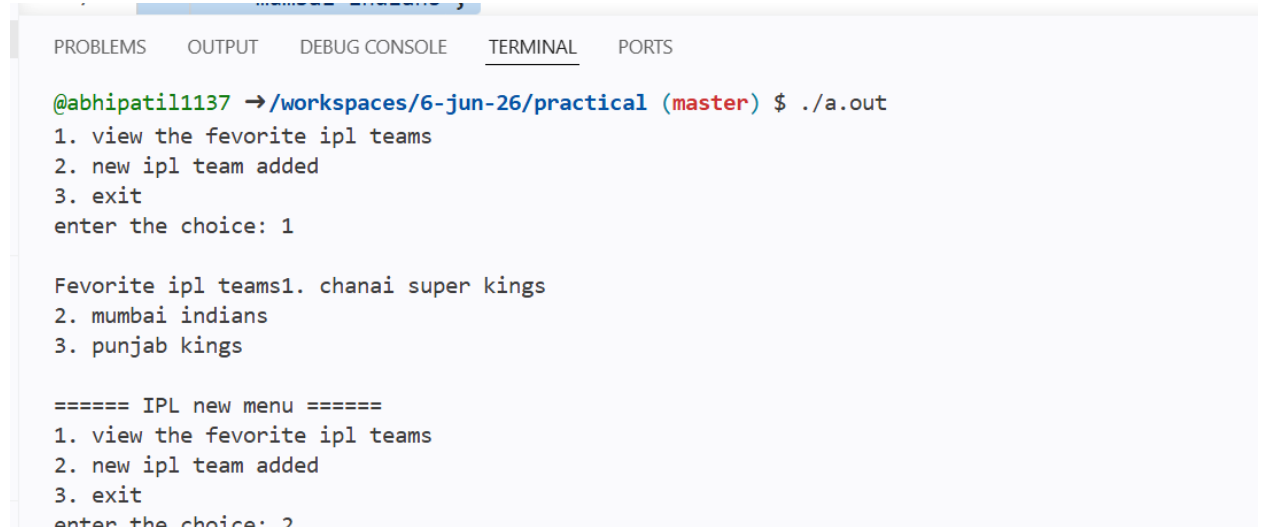
    break;
case 3:
    printf("exite the progaram \n");
    return 0;

default:
    printf("invalid choice plsease try again\n");

}
}
```

```
    return 0;
}
```

Output-



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

@abhipatil1137 → /workspaces/6-jun-26/practical (master) $ ./a.out
1. view the favorite ipl teams
2. new ipl team added
3. exit
enter the choice: 1

Favorite ipl teams1. chanai super kings
2. mumbai indians
3. punjab kings

===== IPL new menu =====
1. view the favorite ipl teams
2. new ipl team added
3. exit
enter the choice: 2
```

- 3) Build a 'Guess the Song' game like Spotify — the program randomly picks a song name from a list and asks the user to guess it. Use a do-while loop so the user can keep guessing until they get it right.

Constraint: Use at least 3 song names of your choice.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>

int main() {
    char songs[3][30] = {
        "Shape of You",
        "Believer",
        "Perfect"
    };

    char guess[30];
    int random;
```

```

// Generate random song
srand(time(0));
random = rand() % 3;

printf("===== Guess the Song Game =====\n");
printf("Available songs are one of these:\n");
printf("- Shape of You\n");
printf("- Believer\n");
printf("- Perfect\n\n");

do {
    printf("Enter your guess: ");
    scanf("%s", guess);

    if (strcmp(guess, songs[random]) == 0) {
        printf("🎉 Correct! You guessed the song.\n");
    } else {
        printf("❌ Wrong guess! Try again.\n");
    }

} while (strcmp(guess, songs[random]) != 0);

return 0;
}

```

Output-

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

@abhipatil1137 → /workspaces/6-jun-26/practical (master) $ ./a.out

Enter your guess: perfect
❌ Wrong guess! Try again.
Enter your guess: believr
❌ Wrong guess! Try again.
Enter your guess: shape of you
❌ Wrong guess! Try again.
Enter your guess: guess
❌ Wrong guess! Try again.
Enter your guess: believer
❌ Wrong guess! Try again.
Enter your guess: Believer
🎉 Correct! You guessed the song.

```

- 4) Explain with your own example the difference between entry-controlled and exit-controlled loops by writing a short code snippet for each (for/while vs do-while) and describing what happens if the loop condition is false at the start.

Entry-Controlled Loop (for, while)

The condition is checked **before** the loop body executes.

If the condition is false at the beginning, the loop body **does not execute**.

Exit-Controlled Loop (do-while)

The condition is checked **after** the loop body executes.

The loop body **executes at least once**, even if the condition is false initially.

Code1-Entry-Controlled Loop (while)

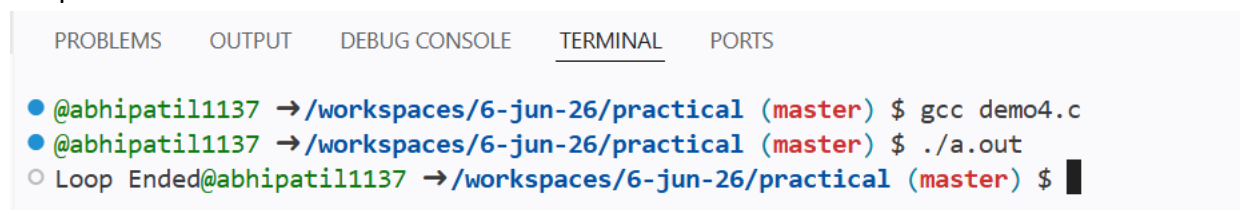
```
#include <stdio.h>

int main() {
    int num = 5;

    while (num < 5) {
        printf("%d\n", num);
        num++;
    }

    printf("Loop Ended");
    return 0;
}
```

Output –



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

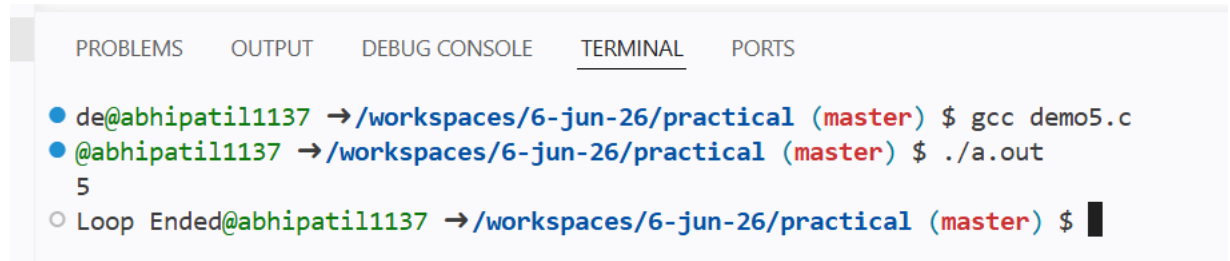
● @abhipatil1137 →/workspaces/6-jun-26/practical (master) $ gcc demo4.c
● @abhipatil1137 →/workspaces/6-jun-26/practical (master) $ ./a.out
○ Loop Ended@abhipatil1137 →/workspaces/6-jun-26/practical (master) $
```

Code-Exit-Controlled Loop (do-while)

```
#include <stdio.h>
```

```
int main() {  
    int num = 5;  
  
    do {  
        printf("%d\n", num);  
        num++;  
    } while (num < 5);  
  
    printf("Loop Ended");  
    return 0;  
}
```

Output-



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
● de@abhipatil1137 → /workspaces/6-jun-26/practical (master) $ gcc demo5.c  
● @abhipatil1137 → /workspaces/6-jun-26/practical (master) $ ./a.out  
5  
○ Loop Ended@abhipatil1137 → /workspaces/6-jun-26/practical (master) $ █
```